Robot Perception and Learning

Model Based RL

Tsung-Wei Ke

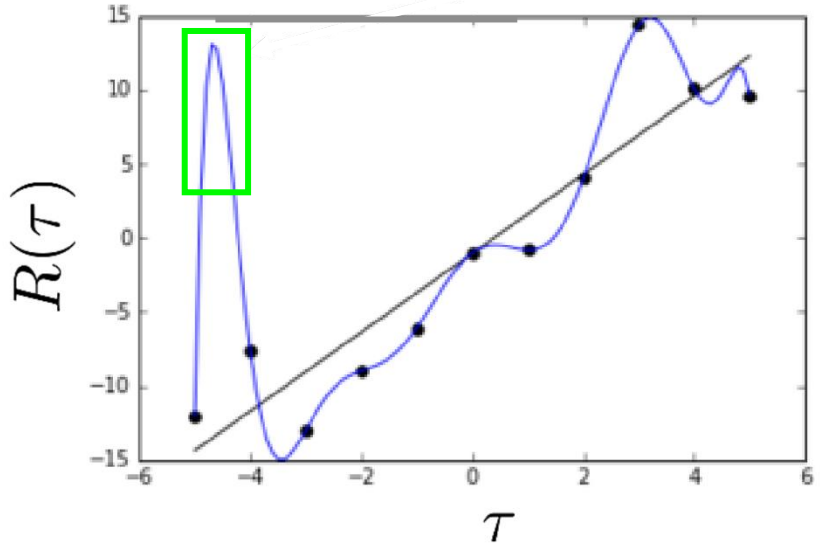
Fall 2026



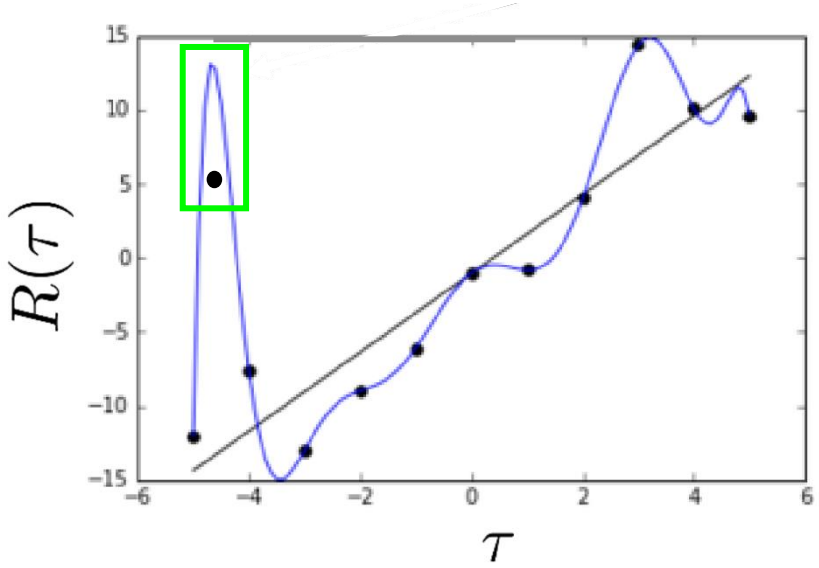
Appendix 1

But Why Pure Model-based RL Doesn't Work?

High-variance reward trajectory induced by overfitting



High-variance reward trajectory induced by underfitting

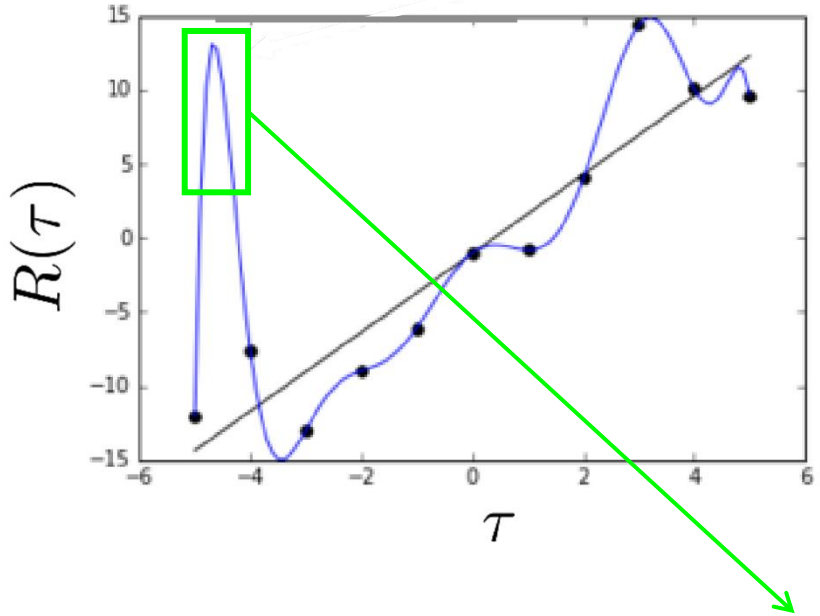


- Incorrect dynamics models lead to inaccurate reward estimates for planned rollouts

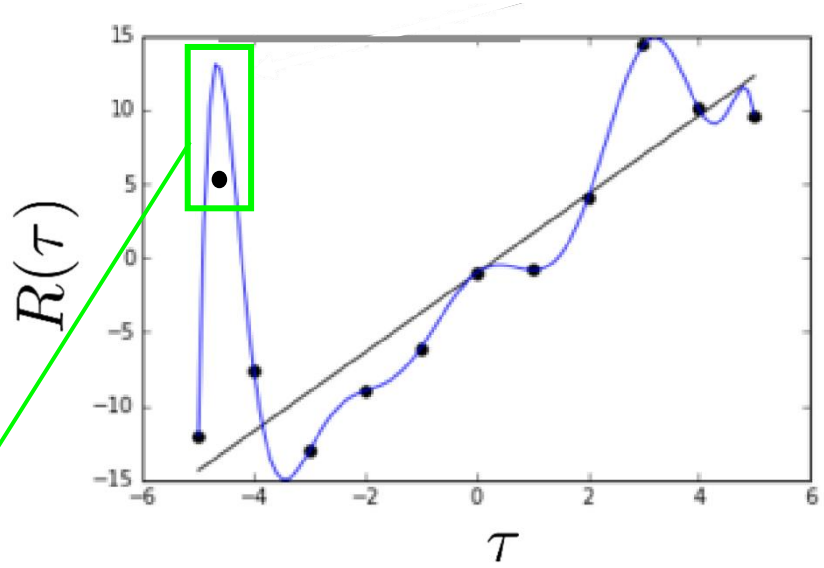
Slide adapted from S. Levine

But Why Pure Model-based RL Doesn't Work?

High-variance reward trajectory induced by overfitting



High-variance reward trajectory induced by underfitting



Model-based RL attempts to go to high-variance reward trajectory, which could have low expected reward instead.

Slide adapted from S. Levine

Idea: Account for Model Uncertainty

- Two types of uncertainty: (1) Epistemic uncertainty, (2) Aleatoric uncertainty

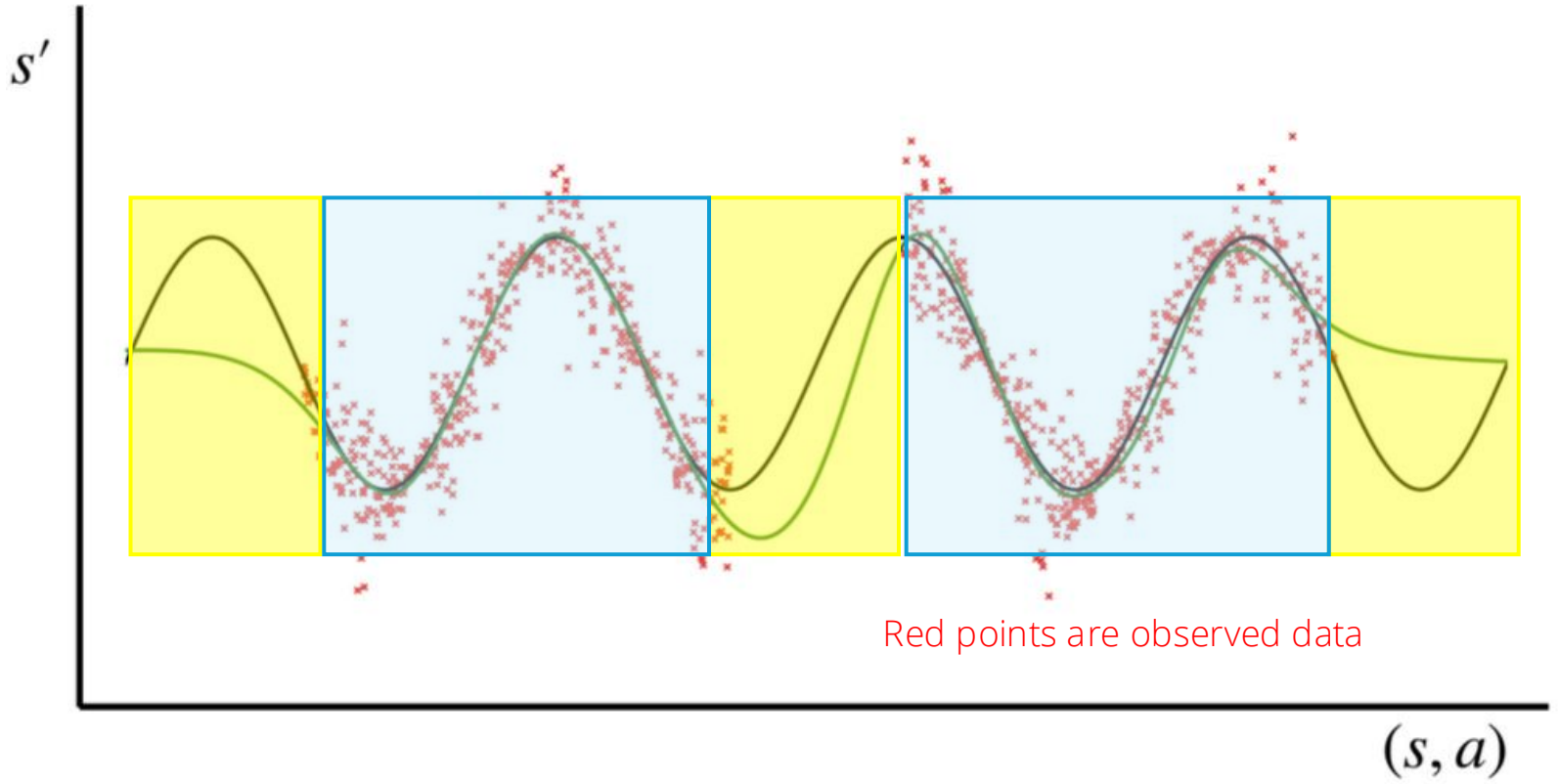


Image credit K. Fragkiadaki

Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models. Chua et al.

Aleatoric Uncertainty

- Aleatoric uncertainty: arises from inherent stochasticity of a system, e.g. observation noise, process noise or multi-modality

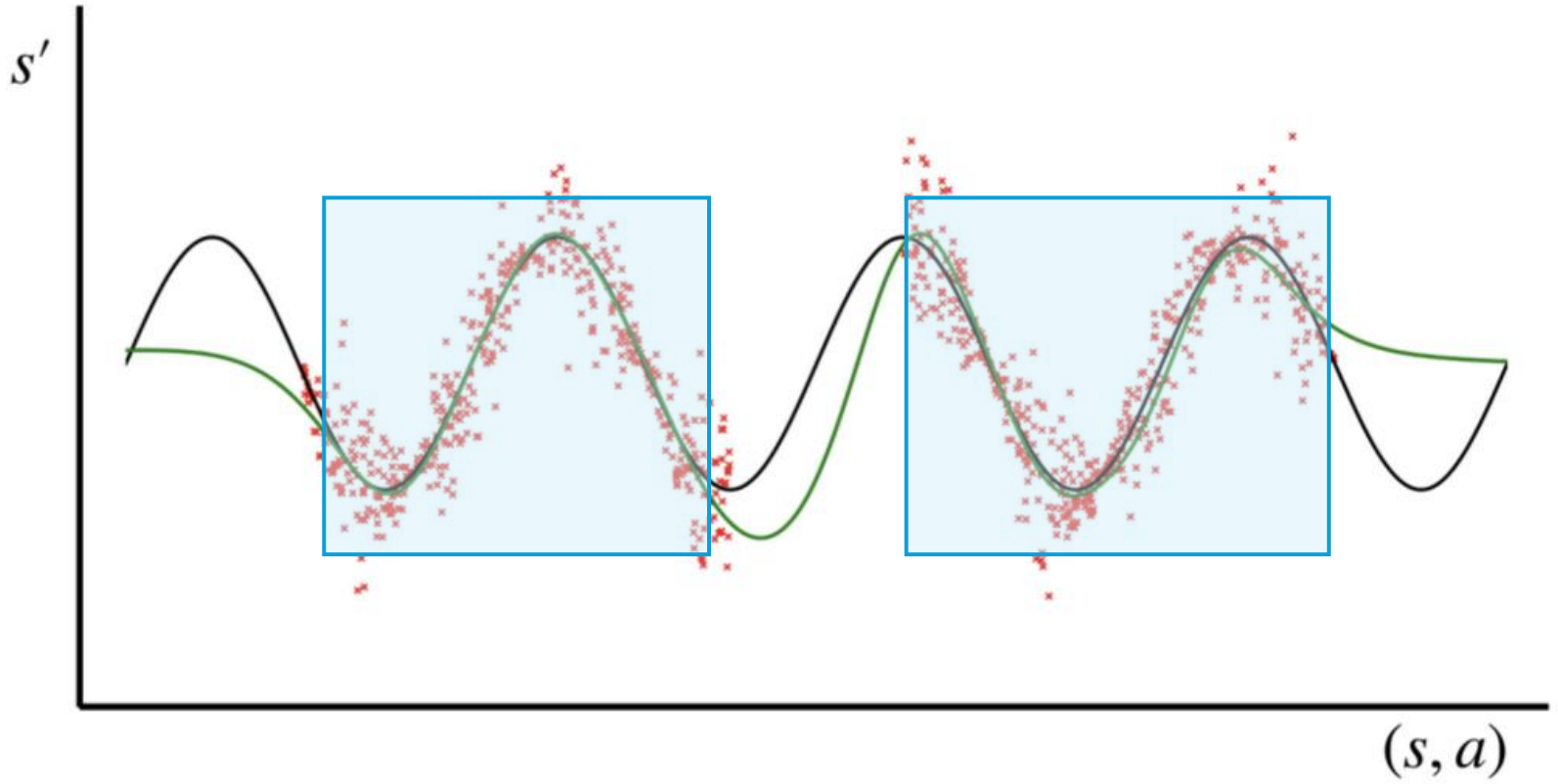


Image credit K. Fragkiadaki

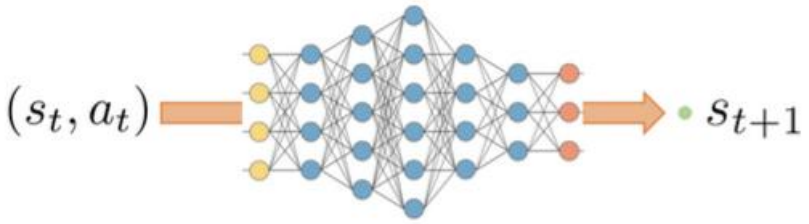
Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models. Chua et al.

Aleatoric Uncertainty

- Aleatoric uncertainty: arises from inherent stochasticity of a system, e.g. observation noise, process noise or multi-modality

- How to model aleatoric uncertainty? If the environment is stochastic:
  - Deterministic regression models fails to capture the output distribution

$$\mathscr{L}_\phi = \sum_{i=1}^{N} \|f(s_i, a_i; \phi) - s_i'\|$$





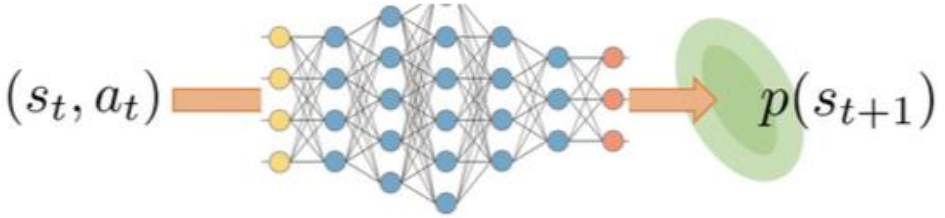
Aleatoric Uncertainty

- Aleatoric uncertainty: arises from inherent stochasticities of a system, e.g. observation noise, process noise or multi-modality

- How to model aleatoric uncertainty? If the environment is stochastic:
  - Deterministic regression models fails to capture the output distribution
  - Probabilistic models can capture the output distribution

- An example that captures Gaussian distribution:

$$p_\phi(s'|s,a) = \frac{\exp\left(-\frac{1}{2}(s'-\mu(s,a;\phi)^\top(\mathbf{\Sigma}(s,a;\phi))^{-1}(s'-\mu(s,a;\phi)\right)}{\sqrt{(2\pi)^d \det\mathbf{\Sigma}(s,a;\phi)}}$$



$$\mathcal{L}_\phi = -\frac{1}{N}\sum_{i=1}^{N} \log p_\phi(s_i'|s_i,a_i)$$

$$= \frac{1}{2}(s_i' - \mu(s_i,a_i;\phi))^T \mathbf{\Sigma}(s_i,a_i;\phi)^{-1}(s_i' - \mu(s_i,a_i;\phi)) + \frac{1}{2}\log(\det\mathbf{\Sigma}(s_i,a_i;\phi)) + \text{ const.}$$

Slide credit K. Fragkiadaki

Epistemic Uncertainty

- Epistemic uncertainty: subjective uncertainty about the dynamics function, due to a lack of sufficient data to uniquely determine the underlying system exactly
- The model is certain about the data, but we are not certain about the model

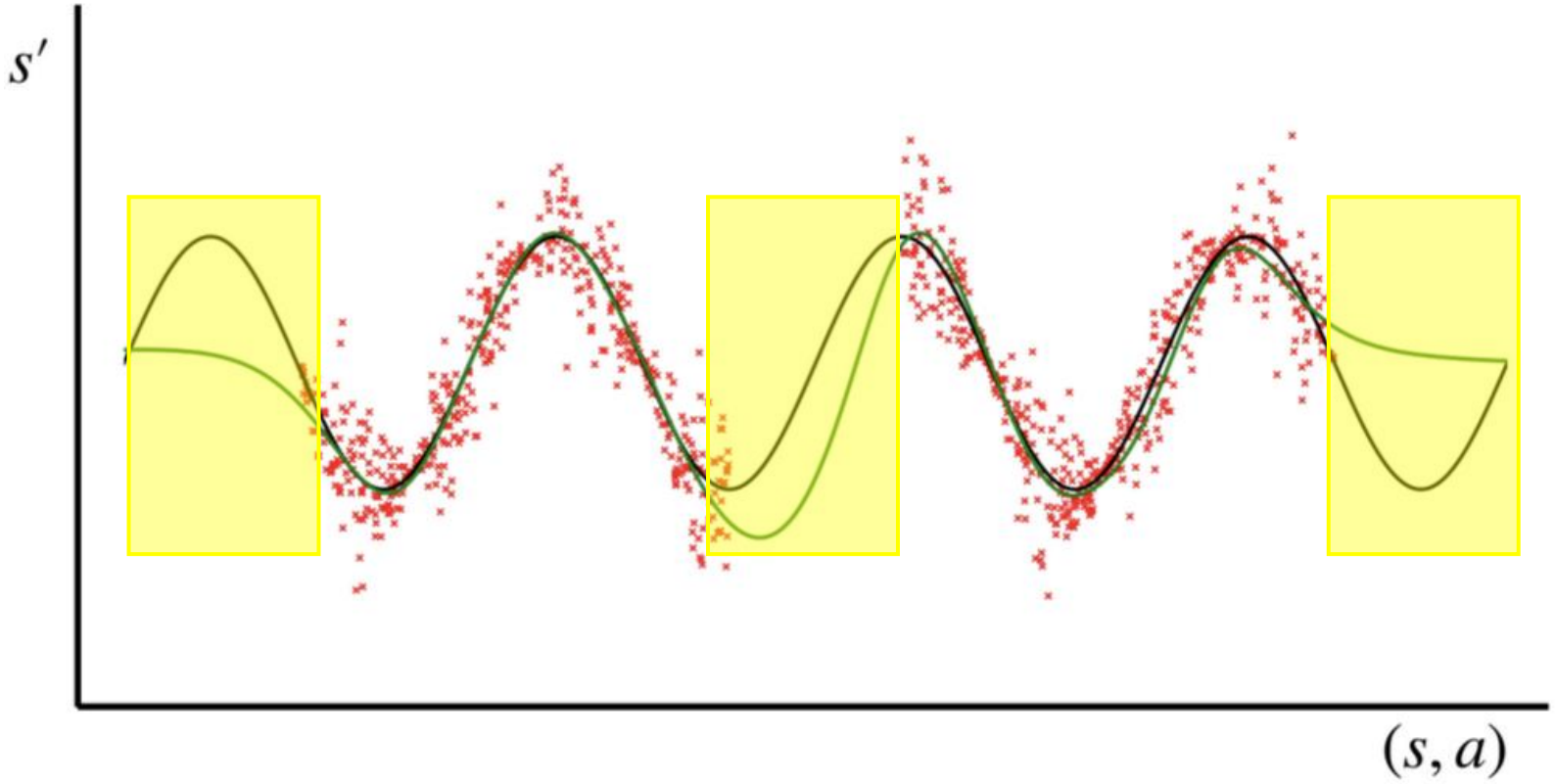


Image credit K. Fragkiadaki

Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models. Chua et al.

How to Model Epistemic Uncertainty? A Bayesian Inference Method

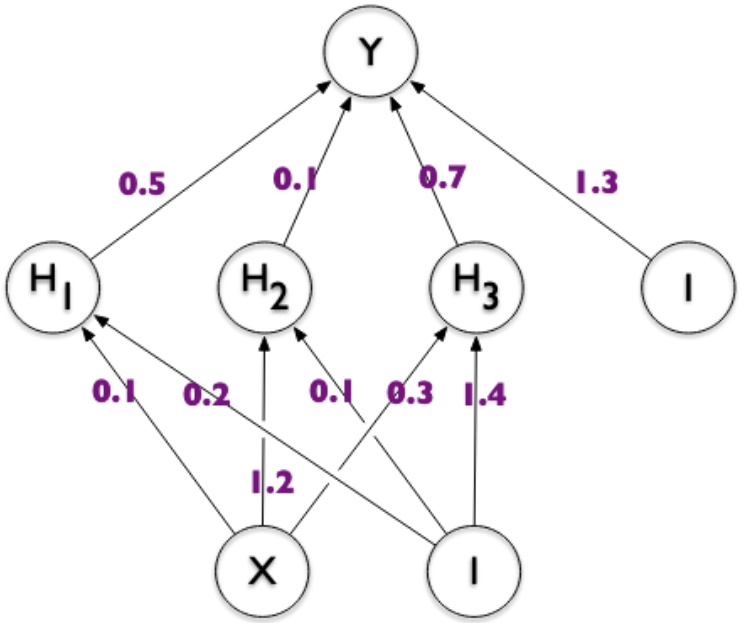
$$P(\text{ hypothesis } | \text{ data }) = \frac{P(\text{ hypothesis })P(\text{ data } | \text{ hypothesis })}{\sum_h P(h)P(\text{ data } | h)}$$

- Hypotheses here are weights for our learning model, i.e., weights of our neural networks that learns the transition dynamics
- The idea is to keep all the hypotheses that fit equally well the training set instead of committing to one, so that I can represent my uncertainty

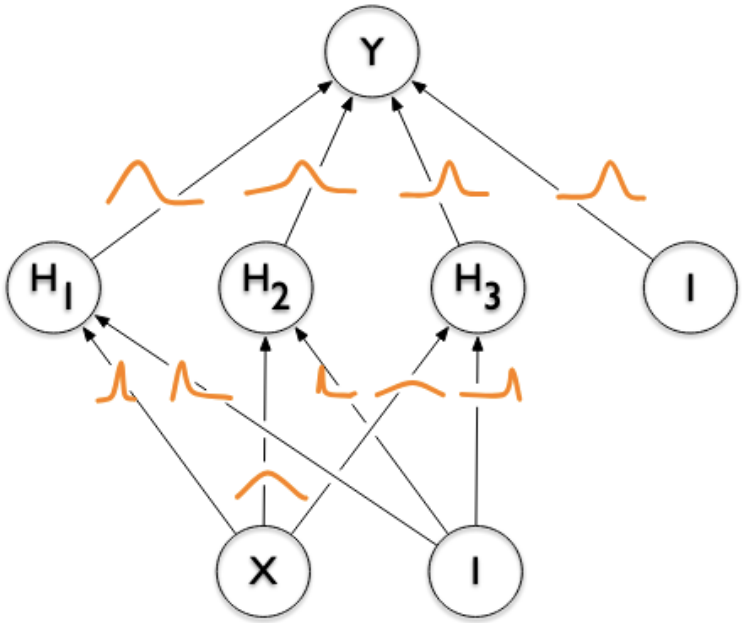
Slide credit K. Fragkiadaki

How to Model Epistemic Uncertainty? A Bayesian Inference Method

Network with fixed weights



Network with a distribution of weights

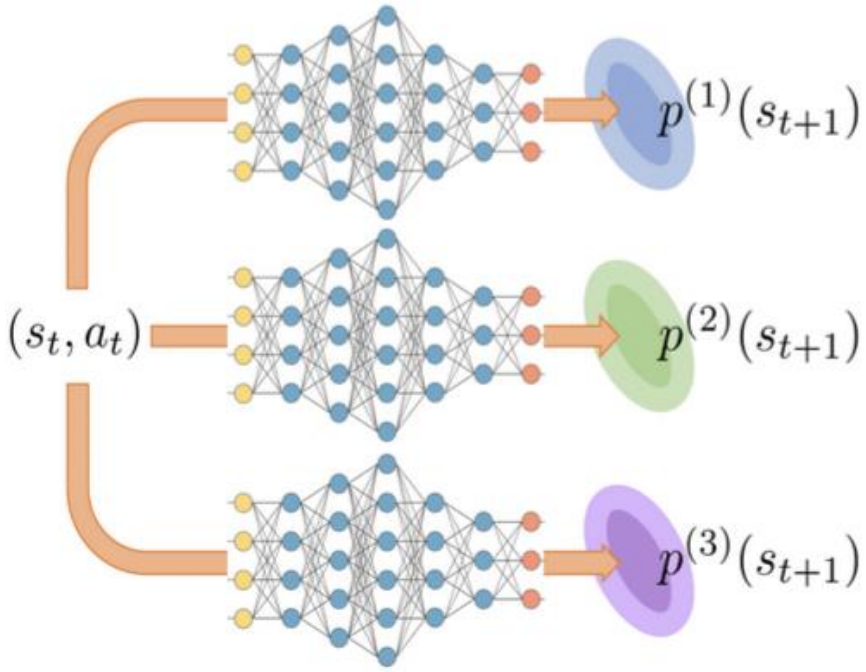


predict according to:

$$\int p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \theta) p(\theta|\mathcal{D}) d\theta$$

Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models. Chua et al.

How to Model Epistemic Uncertainty? An Ensemble-of-Bootstraps Method



- Model ensembles are good approximation of Bayesian Nets:

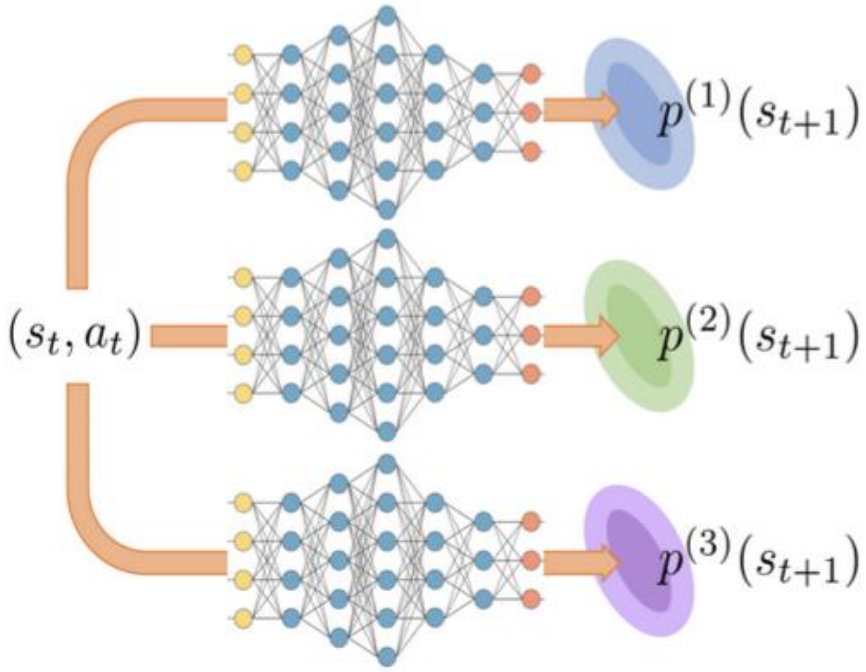
$$p(\theta|D) \approx \frac{1}{N}\sum_i \sigma(\theta_i)$$

$$\int p(s_{t+1}|s_t, a_t, \theta)p(\theta|D)d\theta \approx \frac{1}{N}\sum_i p(s_{t+1}|s_t, a_t, \theta_i)$$

Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models. Chua et al.

Slide adapted from K. Fragkiadaki

How to Model Epistemic Uncertainty? An Ensemble-of-Bootstraps Method



- Model ensembles are good approximation of Bayesian Nets:

$$p(\theta|D) \approx \frac{1}{N}\sum_i \sigma(\theta_i)$$

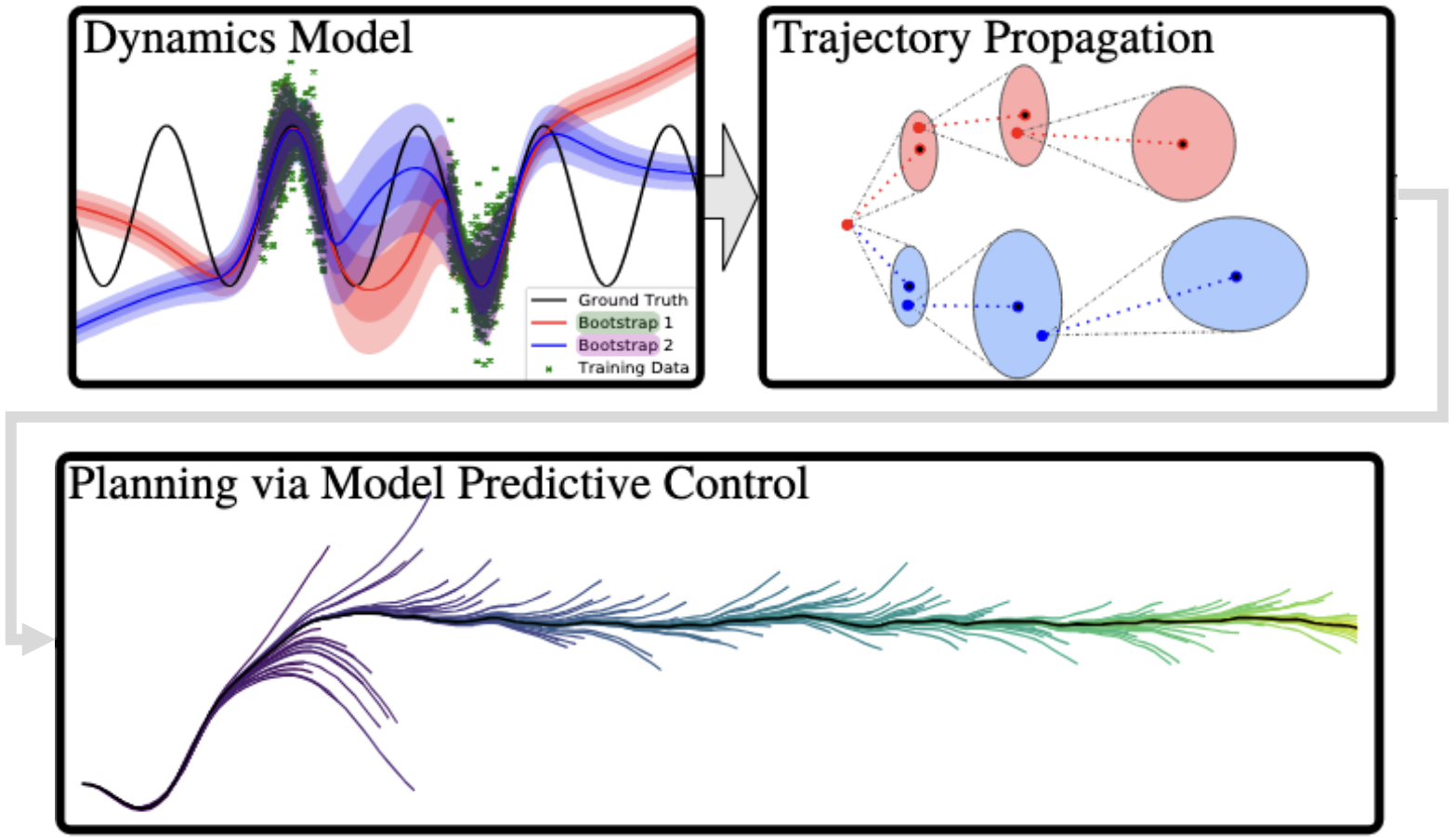
$$\int p(s_{t+1}|s_t, a_t, \theta)p(\theta|D)d\theta \approx \frac{1}{N}\sum_i p(s_{t+1}|s_t, a_t, \theta_i)$$

- Main idea: generate "independent" datasets for each "independent" model:
  - For seen data, they all agree (low-entropy prediction)
  - For unseen data, they disagree (high-entropy prediction)

Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models. Chua et al.

Slide adapted from K. Fragkiadaki

Unrolling an Ensemble of Bootstraps



Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models. Chua et al.

Unrolling an Ensemble of Bootstraps

Algorithm 1 Our model-based MPC algorithm '*PETS*':

1: Initialize data $\mathbb{D}$ with a random controller for one trial.
2: **for** Trial $k = 1$ to K **do**
3: Train a *PE* dynamics model $\widetilde{f}$ given $\mathbb{D}$.
4: **for** Time $t = 0$ to TaskHorizon **do**
5: **for** Actions sampled $\boldsymbol{a}_{t:t+T} \sim \text{CEM}(\cdot)$, 1 to NSamples **do**
6: Propagate state particles $\boldsymbol{s}_\tau^p$ using *TS* and $\widetilde{f}|\{\mathbb{D}, \boldsymbol{a}_{t:t+T}\}$.
7: Evaluate actions as $\sum_{\tau=t}^{t+T} \frac{1}{P} \sum_{p=1}^{P} r(\boldsymbol{s}_\tau^p, \boldsymbol{a}_\tau)$
8: Update CEM$(\cdot)$ distribution.
9: Execute first action $\boldsymbol{a}_t^*$ (only) from optimal actions $\boldsymbol{a}_{t:t+T}^*$.
10: Record outcome: $\mathbb{D} \leftarrow \mathbb{D} \cup \{\boldsymbol{s}_t, \boldsymbol{a}_t^*, \boldsymbol{s}_{t+1}\}$.

Trajectory sampling: the strategy to sample bootstrap as the dynamics model

Results: Performant Model-Based RL

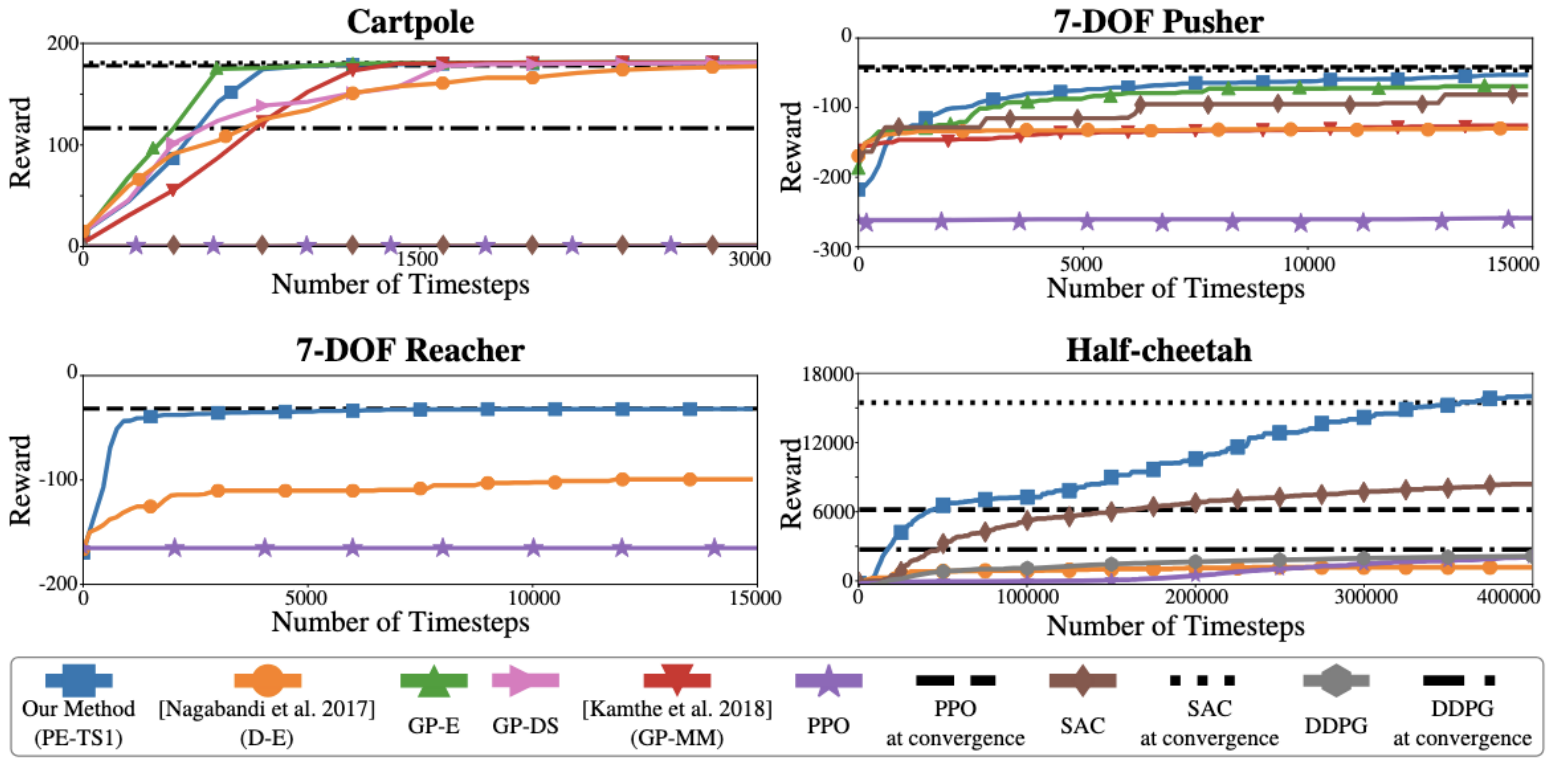


Figure 3: Learning curves for different tasks and algorithm. For all tasks, our algorithm learns in under 100K time steps or 100 trials. With the exception of Cartpole, which is sufficiently low-dimensional to efficiently learn a GP model, our proposed algorithm significantly outperform all other baselines. For each experiment, one time step equals 0.01 seconds, except Cartpole with 0.02 seconds. For visual clarity, we plot the average over 10 experiments of the maximum rewards seen so far.

Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models. Chua et al.